

Vulnerability Assessment Planning Assignment

Name: Gihan Wijesinghe

Date: 2026 April 3rd

Target URL: http://143.198.205.185/

Table of Contents

01. Executive Summary 4

02. Reconnaissance Strategy 5

1. Information Gathering Framework (Phased Approach) 5

2. Legal and Ethical Considerations 5

3. Operational Security (OpSec) 5

4. Intelligence Objectives 6

5. Success Metrics 6

Reconnaissance is complete when no new endpoints are discovered after repeated navigation and source analysis. 6

All HTML source files and visible hyperlinks have been fully inspected. 6

All URL patterns and parameters have been documented. 6

The full attack surface has been mapped without triggering any server-side interaction. 6

03. Tool Selection & Justification 7

A – Browser-Based Observation 7

B – Endpoint & Link Enumeration 7

C – Technology Stack Fingerprinting 7

D – Information Disclosure Documentation 7

E – Historical & Archive Analysis 7

04. Methodology Development 9

Initial Scoping 9

Application Structure and Endpoint Mapping 9

Information Disclosure Identification 10

URL Pattern and IDOR Analysis 10

Intelligence Correlation and Attack Surface Mapping 11

05. Recommendations 13

Secure Administrative Access 13

Secure File Upload Functionality 13

Restrict Network Utility Access 13

Mitigate IDOR Vulnerability 13

Improve Authentication Security *13*

Reduce Information Disclosure *13*

06. Active Assessment Scope *14*

SQL Injection Testing *14*

IDOR Exploitation Testing *14*

Command Injection Testing *14*

File Upload Exploitation Testing *14*

Cross-Site Scripting (XSS) Testing *14*

01.Executive Summary

This report presents the findings of a passive reconnaissance assessment conducted on the web application hosted at http://143.198.205.185/. The objective of this assessment was to identify publicly accessible information, exposed system components, and potential security risks without interacting with or exploiting the application.

The analysis identified several key security concerns. A total of **7 publicly accessible endpoints** were discovered, including critical functionality such as administrative access (/admin.php), user profile access (/profile.php), file upload (/upload.php), and network utilities (/ping.php). Notably, at least **one administrative interface was accessible without authentication**, significantly increasing the risk of unauthorized system access.

The application exposes multiple types of data without requiring authentication. These include **user-related information (profiles and identifiers), product data, and system structure details**. Additionally, the use of predictable URL parameters (e.g., profile.php?id=1) indicates a potential for unauthorized access to user data, representing a high risk to data confidentiality.

From a security perspective, the application demonstrates a **high overall risk posture**. Key risks include weak access control, excessive information disclosure, and exposure of high-risk functionalities such as file upload and command execution features. These issues could allow attackers to gain unauthorized access, manipulate data, or execute malicious actions if not properly secured.

Based on these findings, it is strongly recommended that immediate remediation actions be taken, particularly in enforcing access control, securing sensitive endpoints, and reducing unnecessary data exposure. Further **active security testing** is also recommended to validate the presence and impact of identified vulnerabilities.

02.Reconnaissance Strategy

1. Information Gathering Framework (Phased Approach)

A passive reconnaissance framework is designed to systematically discover all publicly accessible endpoints without interacting with backend logic or triggering server-side actions.

Phase 1 – Navigation Mapping: Explore visible navigation elements such as menus, headers, and footers to record all accessible URLs and page functions.

Phase 2 – HTML Source Analysis: Inspect page source to extract hyperlinks, hidden references, forms, and resource paths.

Phase 3 – Structural Enumeration: Identify directory structures, file extensions, and relationships between pages.

2. Legal and Ethical Considerations

Passive reconnaissance is limited to observing publicly available information without submitting input or manipulating requests.

Active exploitation begins when requests are intentionally crafted to modify system behavior or access restricted data.

Sensitive data such as PII or credentials must be documented only for reporting and must not be used or disclosed.

Accessing publicly available admin panels is ethical during observation, but attempting authentication or bypassing security controls is not permitted.

3. Operational Security (OpSec)

Use private/incognito browsing mode and disable unnecessary browser extensions.

Avoid form submissions, login attempts, or any interaction that triggers server-side processing.

Minimize request frequency to avoid generating detectable traffic patterns.

Store findings securely in offline or encrypted notes to protect sensitive information.

4. Intelligence Objectives

- The objective is to systematically map all publicly accessible endpoints and exposed application structures.
- Identify all unauthenticated data exposure such as usernames, emails, and system messages.
- Analyze URL parameter patterns including (`?id=`, `?user=`,) and (`?page=` for insecure design indicators.
- Identify internal infrastructure exposure such as admin panels, upload systems, and diagnostic tools.

5. Success Metrics

Reconnaissance is complete when no new endpoints are discovered after repeated navigation and source analysis.

All HTML source files and visible hyperlinks have been fully inspected.

All URL patterns and parameters have been documented.
The full attack surface has been mapped without triggering any server-side interaction.

03.Tool Selection & Justification

A – Browser-Based Observation

Browser-based tools like Developer Tools and View Source are used to inspect HTTP headers, HTML content, and URL structures directly. They work through normal browsing behavior, making them completely passive and undetectable. This method is effective for understanding the frontend structure of the application. However, it is limited to visible content and requires manual effort.

B – Endpoint & Link Enumeration

Endpoints are identified by following visible links and extracting URLs from the HTML source code. Passive tools like Burp Suite (without interception) help observe requests without modifying them. This ensures that only existing application paths are explored without triggering alerts. However, hidden or unlinked endpoints may not be discovered.

C – Technology Stack Fingerprinting

Tools such as Wappalyzer and BuiltWith analyze HTTP headers, file extensions, and response patterns to identify technologies used in the application. This provides useful insights into the backend environment. Since the analysis is passive, it does not generate suspicious traffic. However, the results may sometimes be incomplete or inaccurate.

D – Information Disclosure Documentation

Documentation tools like Word, Obsidian, and screenshots are used to systematically record findings such as endpoints and exposed data. This process is entirely offline and does not interact with the target system. It ensures organized reporting and analysis. The main limitation is the risk of improper handling of sensitive information.

E – Historical & Archive Analysis

Archive tools like Wayback Machine and Google Cache are used to retrieve historical or cached versions of websites. They can reveal previously exposed content and endpoints. These tools operate without directly interacting with the target. However, they are often ineffective for bare IP targets, as such systems are rarely indexed.

Category	Tools	Justification (Why Used)	How They Avoid Detection	Limitations
A - Browser-Based Observation	Chrome DevTools, Firefox DevTools, View Page Source	Used to inspect HTTP headers, HTML structure, and URL formats directly from the browser without modifying requests	Operates as normal user traffic (no payloads, no automation)	Manual process, limited visibility into backend logic
B - Endpoint & Link Enumeration	Manual Navigation, View Source (hrefextraction), Burp Suite (passive mode)	Helps identify all accessible endpoints via links, forms, and URL patterns without brute forcing	Only follows existing links, no forced browsing or fuzzing	May miss hidden or unlinked endpoints, time-consuming
C - Technology Stack Fingerprinting	Wappalyzer, BuiltWith, HTTP Headers (DevTools)	Identifies backend technologies using file extensions (.php), headers, and response behavior	Passive fingerprinting using already available data	May be inaccurate, cannot detect custom or hidden technologies
D - Information Disclosure Documentation	Obsidian / Notepad / Word, Screenshots, Local Secure Storage	Used to systematically record findings such as endpoints, exposed data, and observations for reporting	No interaction with target system; purely offline documentation	Risk of insecure storage if not handled properly
E - Historical & Archive Analysis	Wayback Machine, Google Cache, Netcraft	Retrieves historical data, cached pages, and previously exposed endpoints (if domain exists)	No direct interaction with target; uses third-party archives	Limited or no data for bare IP targets, may be outdated

04.Methodology Development

Initial Scoping

The assessment began by confirming the target URL and recording the date and time of access, as bare IP environments may change ownership or configuration over time. The scope of "publicly accessible" was defined as any resource reachable through standard navigation or visible within the HTML source code without authentication or input submission.

A clean browser environment was prepared using incognito mode to avoid cached data and stored sessions. Developer tools were enabled to monitor network activity and inspect page structure. Upon loading the homepage, key details were recorded, including the application name (*VulnApp Lab*), its purpose as a security training platform, and explicit disclosures of intentionally vulnerable features.

Application Structure and Endpoint Mapping

The navigation menu was used as the primary source for endpoint enumeration. All visible links were recorded, including `/index.php`, `/login.php`, `/products.php`, `/search.php`, `/profile.php`, `/admin.p hp`, `/ping.php`, and `/upload.php`.

Each endpoint was visited individually, and the following details were documented:

- Full URL structure
- Page title and functionality
- Types of data displayed (e.g., user information, product listings)

HTML source code was inspected for each page to identify hidden links, comments, and form actions. Additional endpoints and internal references were extracted from `href` attributes and commented sections.

HTTP response headers were captured using the browser's developer tools, including server type, content type, and any technology indicators such as `X-Powered-By`. All discovered resources and URLs were consolidated into a structured list for further analysis.

Information Disclosure Identification

The analysis focused on identifying all information exposed without authentication:

- The homepage (`/index.php`) revealed the application's purpose and explicitly listed multiple vulnerabilities, indicating intentional security weaknesses.
- The products page (`/products.php`) displayed product-related data, including names and pricing, with potential indicators of restricted or internal items.
- The admin panel (`/admin.php`) exposed structured user data, including roles and other attributes, which were classified based on sensitivity.
- The profile page (`/profile.php`) displayed user-specific information, including personally identifiable data, accessible via a numeric parameter pattern.
- The login page (`/login.php`) was tested with dummy credentials to observe system responses and error handling behavior.

Across all pages, internal paths, system messages, and structural details were recorded as potential indicators of backend configuration.

URL Pattern and IDOR Analysis

URL structures were analyzed to identify patterns indicating insecure design. The profile page utilized a parameterized structure (`/profile.php?id=1`), suggesting direct object access based on user-controlled input.

The sequential nature of the parameter indicated a potential Insecure Direct Object Reference (IDOR) vulnerability, where user data could be accessed by modifying the identifier. The presence of the admin panel within the navigation menu suggested weak access control enforcement.

Other endpoints were reviewed for similar parameter patterns, and all findings were classified based on their potential security impact. The exposure of user-related data through predictable identifiers was categorized as a high-risk issue.

Intelligence Correlation and Attack Surface Mapping

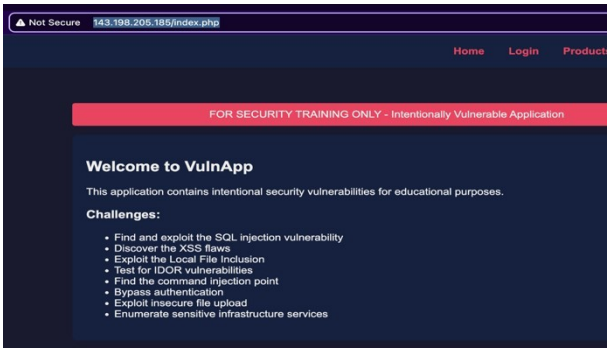
All findings were consolidated to produce a comprehensive attack surface overview. Key endpoints were mapped to their respective functionalities and associated risks.

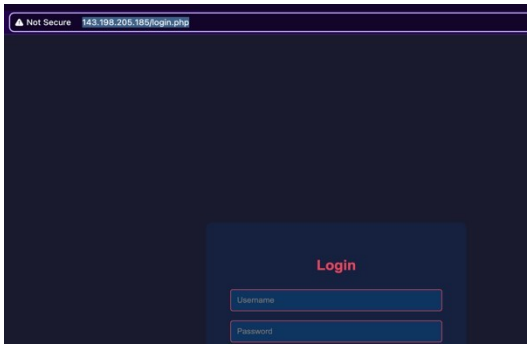
The vulnerabilities disclosed on the homepage, including SQL injection, cross-site scripting (XSS), local file inclusion (LFI), IDOR, and command injection, were cross-referenced with passive findings. Certain risks, such as IDOR and command injection (via the network tool), were identifiable through observation alone.

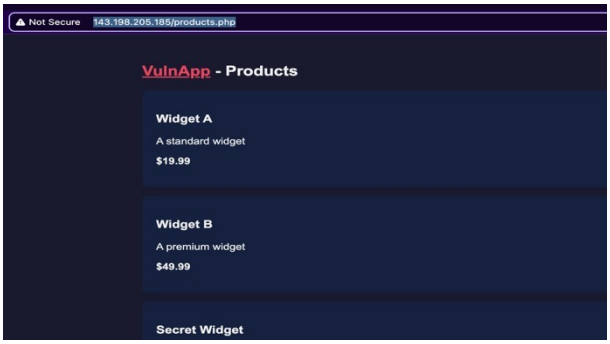
Each finding was mapped to relevant OWASP Top 10 categories, including Broken Access Control, Injection, and Security Misconfiguration. Risk levels were assigned as follows:

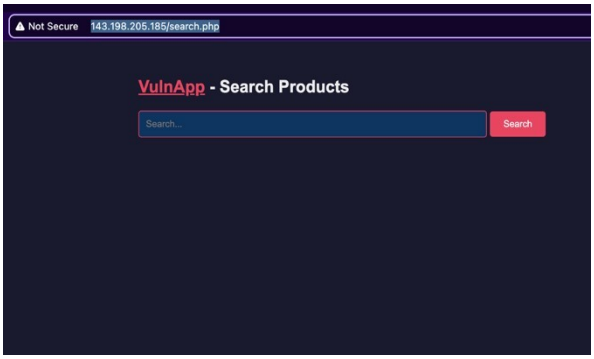
- Critical: Admin panel exposure, file upload functionality, network tool
- High: Profile data access, authentication mechanisms
- Medium: Information disclosure
- Low: Structural and design observations

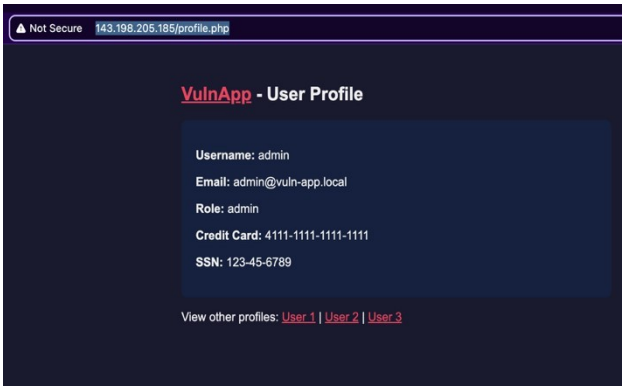
Findings were further categorized into those confirmed through passive analysis and those requiring active testing. Based on this, key testing priorities were identified, including SQL injection, IDOR exploitation, command injection, file upload abuse, and XSS testing.

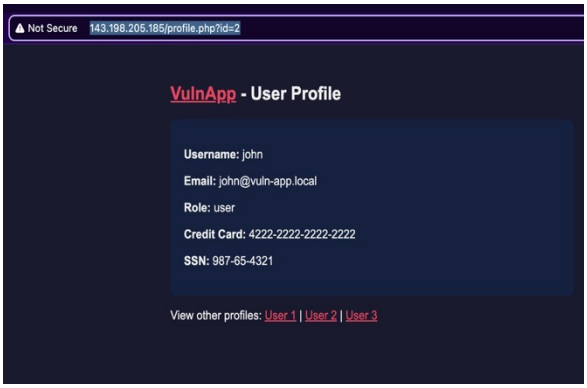


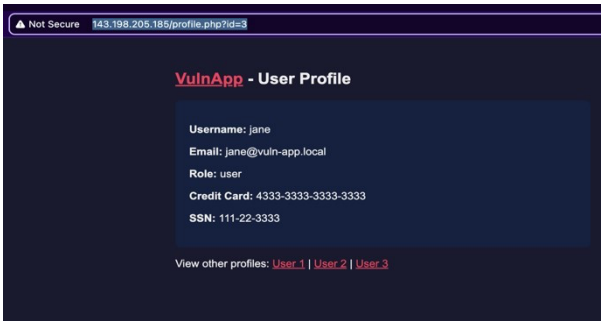


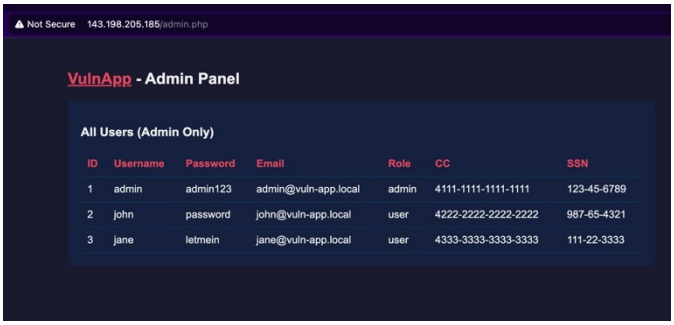


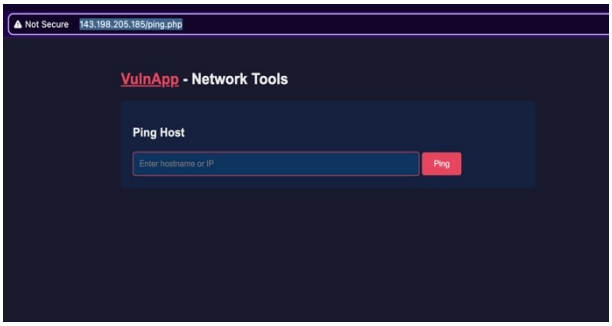


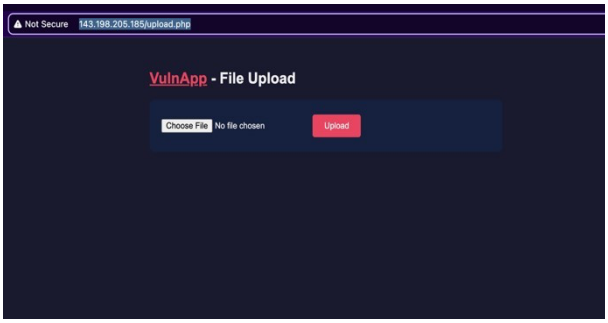

Not Secure 143.198.205.185/profile.php
VulnApp - User Profile
Username: admin
Email: admin@vuln-app.local
Role: admin
Credit Card: 4111-1111-1111-1111
SSN: 123-45-6789
View other profiles: User 1 | User 2 | User 3


Not Secure 143.198.205.185/profile.php?id=2
VulnApp - User Profile
Username: john
Email: john@vuln-app.local
Role: user
Credit Card: 4222-2222-2222-2222
SSN: 987-65-4321
View other profiles: User 1 | User 2 | User 3


Not Secure 143.198.205.185/profile.php?id=3
VulnApp - User Profile
Username: jane
Email: jane@vuln-app.local
Role: user
Credit Card: 4333-3333-3333-3333
SSN: 111-22-3333
View other profiles: User 1 | User 2 | User 3


Not Secure 143.198.205.185/admin.php
VulnApp - Admin Panel
All Users (Admin Only)
ID Username Password Email Role CC SSN
1 admin admin123 admin@vuln-app.local admin 4111-1111-1111-1111 123-45-6789
2 john password john@vuln-app.local user 4222-2222-2222-2222 987-65-4321
3 jane letmein jane@vuln-app.local user 4333-3333-3333-3333 111-22-3333


Not Secure 143.198.205.185/ping.php
VulnApp - Network Tools
Ping Host
Enter hostname or IP
Ping


Not Secure 143.198.205.185/upload.php
VulnApp - File Upload
Choose File No file chosen
Upload

05.Recommendations

Secure Administrative Access

The administrative interface (`/admin.php`) was found to be publicly accessible without authentication. It is recommended to implement strong authentication mechanisms and rolebased access control (RBAC) to ensure that only authorized users can access administrative functionalities. This will mitigate risks related to unauthorized system control and data exposure.

Secure File Upload Functionality

The file upload endpoint (`/upload.php`) is exposed and may allow users to upload arbitrary files. It is recommended to enforce strict file validation, including file type restrictions, size limitations, and server-side content inspection. Additionally, uploaded files should be stored outside the web root to prevent direct execution.

Restrict Network Utility Access

The network tool (`/ping.php`) is publicly accessible and may accept user input for execution. It is recommended to either disable this functionality or restrict access to authenticated users. Input validation and sanitization must also be implemented to prevent command injection attacks.

Mitigate IDOR Vulnerability

The profile endpoint (`/profile.php?id=`) uses a sequential numeric parameter, which indicates a potential Insecure Direct Object Reference (IDOR) vulnerability. It is recommended to implement proper authorization checks to ensure users can only access their own data. Additionally, indirect identifiers such as tokens should be used instead of predictable IDs.

Improve Authentication Security

The login functionality (`/login.php`) may expose system responses through error messages. It is recommended to use generic error messages that do not reveal internal system behavior. Furthermore, implementing account lockout mechanisms and rate limiting will reduce the risk of brute-force attacks.

Reduce Information Disclosure

The products page (`/products.php`) exposes data without authentication. It is recommended to limit publicly visible information and ensure that sensitive or internal data is only accessible to authenticated users.

06.Active Assessment Scope

SQL Injection Testing

Input fields observed in `/login.php` and `/search.php` suggest that user input is processed by the backend. Therefore, SQL injection testing is recommended to determine whether input validation is properly implemented.

IDOR Exploitation Testing

The sequential parameter pattern in `/profile.php?id=` indicates a high likelihood of IDOR vulnerability. Active testing should be conducted by modifying the parameter values to verify unauthorized data access.

Command Injection Testing

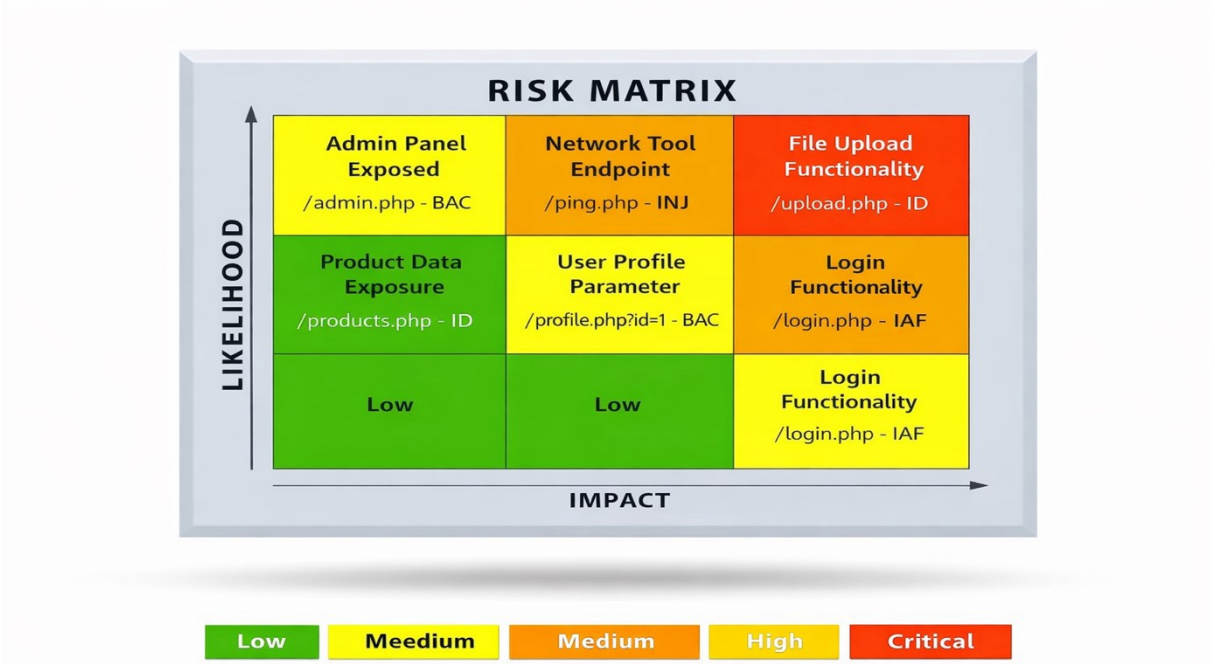
The presence of the `/ping.php` endpoint suggests that user input may be executed as system commands. Active testing is required to verify whether input sanitization is in place.

File Upload Exploitation Testing

The `/upload.php` endpoint should be tested to determine whether it allows malicious file uploads, including executable scripts. This will confirm whether proper validation mechanisms are implemented.

Cross-Site Scripting (XSS) Testing

User input fields across the application indicate potential XSS risks. Active testing should be conducted to determine whether input is properly sanitized and output is safely rendered.


RISK MATRIX
LIKELIHOOD
IMPACT
Admin Panel Exposed
/admin.php - BAC
Network Tool Endpoint
/ping.php - INJ
File Upload Functionality
/upload.php - ID
Product Data Exposure
/products.php - ID
User Profile Parameter
/profile.php?id=1 - BAC
Login Functionality
/login.php - IAF
Low
Low
Login Functionality
/login.php - IAF
Low
Meedium
Medium
High
Critical